

前言

近年来，**DevOps** 在前端领域逐渐火起来，其最大特性是 **自动化**，终极目标就是解放双手。**DevOps**是一组过程、方法和系统的统称，是 **Development**开发与 **Operations**运维 的简写，用于促进技术开发、运营策划和质量保障三个部门间的沟通、协作和整合。

目前而言，**DevOps** 是一种重视软件开发人员与运维技术人员间沟通合作的文化、运动或惯例。透过 **自动化** 软件交付与架构变更的流程，使 **构建、测试、打包、发布和部署** 变得更快捷、频繁和可靠。它的出现是因为软件行业日益清晰地认识到：为了按时交付软件产品与服务，开发与运维的工作必须紧密合作。整个工作流程都是趋向 **半自动化**，甚至是 **全自动化**。

说到 **DevOps** 就必须关联上 **CI/CD**。**DevOps** 与 **CI/CD** 紧密相关，是理论与实践的结合，**DevOps** 要实现人员一体化，就必须借助 **CI/CD** 自动化整个流程。说到这，**CI/CD** 是什么？

持续集成

持续集成又称 **Continuous Integration**，简称 **CI**，指多名开发者在开发不同代码时可频繁地将代码合且互相不影响工作。

持续部署

持续部署又称 **Continuous Deployment**，简称 **CD**，指基于某种工具或平台实现 **代码自动化** 的 **构建、测试、打包、发布和部署**。**持续部署** 在某种程度上代表一个开发团队的更新迭代速率。

持续交付

持续交付又称 **Continuous Delivery**，简称 **CD**，指在 **持续部署** 的基础上将产品交付到线上环境以实现交付高质量的产品。**持续交付** 是产品价值的一种交付，是产品价

值的一种盈利实现。

一个完整的 CI/CD 包括一个 CI 与两个 CD。通过 CI/CD 流水线，软件研发可实现从 构建、测试、打包、发布和部署 甚至整个工程都在流水线中向前推进。一旦将软件研发实施流水线，就可将其 部分工作流程 或 全部工作流程 自动化，以加快开发流程并减少错误。简而言之，CI/CD 提供的流水线操作使企业更轻松地应对软件的自动快速持续交付。

通过一章内容将整个 CI/CD 流水线讲述完毕是很不实际的事情，因此本章将基于前面十几章积累下来的示例做一个最终的 CI 操作，剩下的 CI/CD 内容将在后续掘金课程《从0到1落地前端架构设计》中接着讲述。本章将带领你**基于Github提供的持续集成服务部署一个静态站点**，熟悉 Github Actions 与 Github Pages 的搭配使用，编写配置文件实现提交代码时执行 CI 操作。

背景：哪些基建流程需部署CI

回想下，经过一段时间的学习，已完成了 个人服务器、类库模块、多包仓库、私有仓库 和 文档站点 的搭建与部署。以下将逐一分析，看看哪些基建需部署 CI。

- 个人服务器 作为承载 CI 流水线的底层支持：必须部署
- 类库模块 开发完毕最终会发布到 Npm公有仓库 或 Npm私有仓库，其发布过程可考虑使用 CI：考虑部署
- 多包仓库 与 类库模块 的情况相似，其发布过程可考虑使用 CI：考虑部署
- 私有仓库 开发完毕基本不会发生很大变动，它只是作为一个中继托管仓库：无需部署
- 文档站点 在 类库模块 或 多包仓库 的版本发生迭代后，必定会更新网站内容：必须部署

考虑到 类库模块 与 多包仓库 的发布操作的准确性与安全性必须要有稳定的保障，手动发布可能更稳妥，因此也不考虑部署 CI 了。

第5章选购的服务器只有 1核2G，部署 CI/CD 需搭建 Gitlab，但参照 Gitlab官方 对服务器硬件要求是必须在 2核2G 以上。我尝试在 1核2G 的服务器上搭建 Gitlab，运行起来不是内存爆掉就是进程崩溃，因此不建议不符合要求的服务器强行搭建 Gitlab。若选购的服务器满足条件，可参照掘金课程《从零到一实现一套CI/CD流

程》自行搭建。若选购的服务器不满足条件，也无需担心，可选择 Github 提供的免费持续集成服务 Github Actions 代替。

 掘金小册

基于容器化技术的 CI/CD 流程，助力研发提效

从 0 到 1 实现一套 CI/CD 流程



王圣松

码云 Gitee 研发团队前端开发工程师

你将获得

搭建一套完整的 CI/CD 流程

使用 Jenkins 构建 Docker 镜像

基于 Nexus 搭建私有镜像库

部署 Kubernetes 集群并使用其部署应用镜像



JowayYoung 邀你来学~

现价：¥29.9





当 **类库模块** 或 **多包仓库** 的版本发生迭代后，就会更新这些版本对应功能的文档内容。更新完毕提交到 **Github仓库**，会触发 **Webhooks**，通过 **Github Actions** 监听指定的 **Webhooks** 就能触发提前编写好的脚本。

为了让 **文档站点** 实现国内外访问分流，国内访问指向服务器部署的 **CDN**，国外访问指向 **Github** 部署的 **Github Pages**。该方案的实现，当然是使用 **Nginx** 判断了。

Github Pages 是 **Github** 提供的免费静态站点服务。

概念：Github Actions是什么

Github Actions是基于 **Github** 的 **持续集成服务**。其提供一台虚拟 **服务器实例**，在该实例中允许通过自定义 **Actions** 执行一个或多个命令，以自动地操作代码、构建代码、测试代码、打包代码、发布代码、部署代码、登录远程服务器等。

自定义 **Actions** 如何编写？**Github官方** 的[Marketplace](#)收集了大量官方与第三方 **Actions**，开发者可自由使用里面的 **Actions**，也可提交自己的 **Actions**。不过 **Marketplace** 提供的第三方 **Actions** 几乎能满足日常使用，因此挑选与组合里面的 **Actions** 就足够了。

概念术语

一个完整的 **Actions** 由以下部分组成。

- ☑ **workflow**：工作流程，一个完整且单独运行的 **持续集成服务**
- ☑ **job**：任务，一个或多个 **job** 组成一个 **workflow**，一次 **持续集成服务** 的运行可完成一个或多个任务
- ☑ **step**：步骤，一个或多个 **step** 组成一个 **job**，一个任务的运行由一个或多个步骤根据顺序完成
- ☑ **action**：动作，一个或多个 **action** 组成一个 **step**，一个步骤的运行由一个或多个动作根据顺序执行

配置文件

Github Actions 的配置文件是 workflow文件，必须存放到 .github/workflows 目录中并以 <name>.yaml 的形式命名。

workflow文件 可创建多个，文件名称可根据集成服务的功能任意命名，但后续必须使用 .yaml。当提交代码到 Github仓库 后，只要发现 .github/workflows/<name>.yaml 文件，就会自动挨个文件执行，直至处理完毕所有集成服务。

以下列举一些配置文件的常见字段。

name 表示工作名称，若不设置默认为 workflow文件 的文件名称。若手动完成一个工作流程，会根据顺序执行 checkout检出、 build构建 和 deploy部署，因此将工作名称合并简称为 CBD。

```
name: CBD # 动作名称
```

yaml

on 表示触发事件，上述提到的 Webhooks 可定义一个或多个 Webhooks，通常是 push 与 pull_request。Webhooks 要指定操作的分支，通常是 master 或 main。

```
on: # 触发条件：在push到main分支后
  push:
    branches:
      - main
```

yaml

jobs 表示任务列表，使用对象表示，对象属性表示任务名称，会在 Actions 的执行时显示。

- ✓ **name**：任务名称
- ✓ **runs-on**：虚拟机环境，可选 ubuntu-latest/windows-latest/macos-latest
- ✓ **needs**：执行任务的依赖顺序
- ✓ **steps**：执行步骤，每个任务可将需执行的内容划分为不同步骤
- ✓ **name**：步骤名称
- ✓ **uses**：官方与第三方 Actions
- ✓ **with**：Actions 的入参
- ✓ **run**：执行命令
- ✓ **env**：环境变量

```
jobs: # 任务
  cbd: # 任务ID
```

yaml

```
name: doc cbd # 任务名称
runs-on: ubuntu-latest # 虚拟机环境
steps: # 执行步骤
  # 拉取代码
  - name: Checkout
    uses: actions/checkout@v2
  # 打包文件
  - name: Build
    run: yarn && yarn run deploy
    env:
      AUTHOR: JowayYoung
      AGE: 29
      SEX: male
  - ...
```

方案：基于Github提供的持续集成服务部署一个静态站点

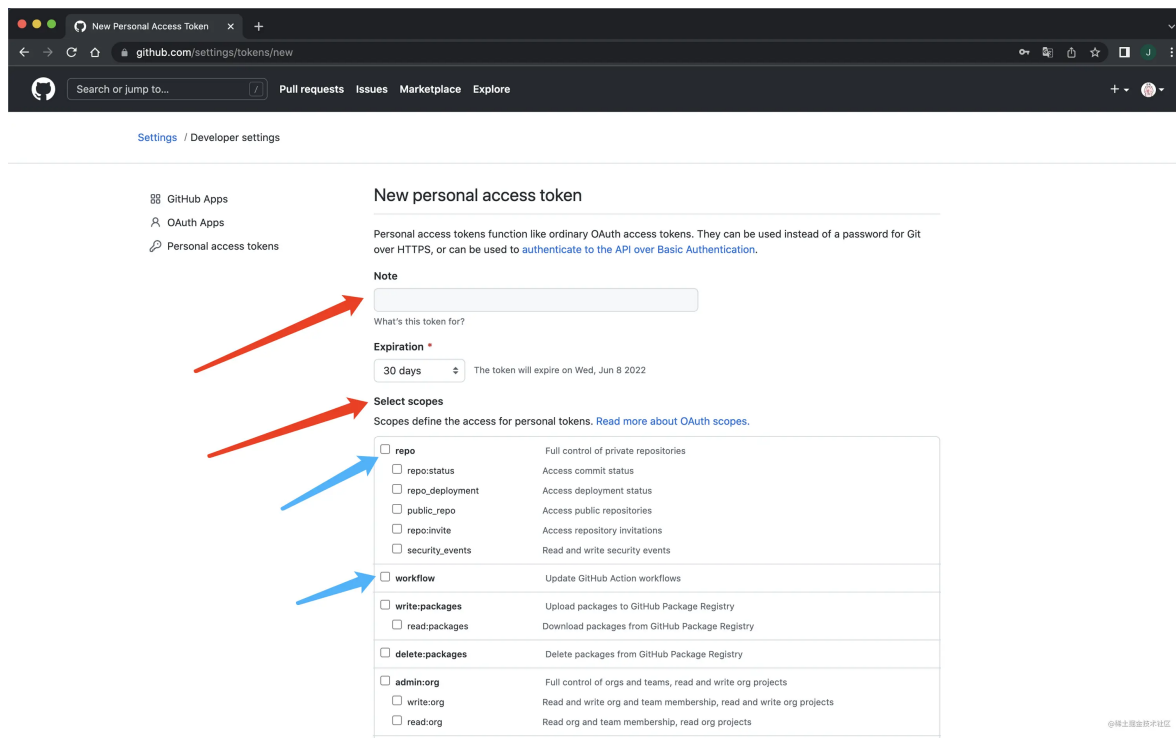
若使用手动部署 [文档站点](#)，首先检出代码，再执行 `yarn` 安装依赖，再执行 `yarn run deploy` 生成静态文件，再将文件提交到 `gh_pages` 分支上，最后通过 [Github Pages](#) 自动生成静态站点。

了解 [Github Actions](#) 后，接着将基于上述步骤通过 [Github Actions](#) 逐步实现可国内外访问分流的 [文档站点](#)。

Github Token与Actions Secret

首先创建**Github Token**。打开[Github设置](#)，点击 `Developer settings`，再点击 `Personal access tokens`，再点击 `Generate new token`。

在 `Note` 中输入备注信息，例如 `GA_TOKEN`。在 `Select scopes` 中选择 `repo` 与 `workflow`。



确定无误后点击 **Generate token**，生成一个 **Github Token**。

Personal access tokens

[Generate new token](#)[Revoke all](#)

Tokens you have generated that can be used to access the [GitHub API](#).

Make sure to copy your personal access token now. You won't be able to see it again!

✓ ghp_hPeoonwIEDLwannpodbE3tMz0ErNQm2JZYhm [Copy](#)

[Delete](#)

Personal access tokens function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#). @稀土掘金技术社区

不要刷新网页，立即点击复制按钮把该 **token** 复制并保存下来，因为刷新网页或重新进入网页，该 **token** 会消失，只剩下 **token名称**。

Personal access tokens

[Generate new token](#)[Revoke all](#)

Tokens you have generated that can be used to access the [GitHub API](#).

GA_TOKEN — repo

Never used

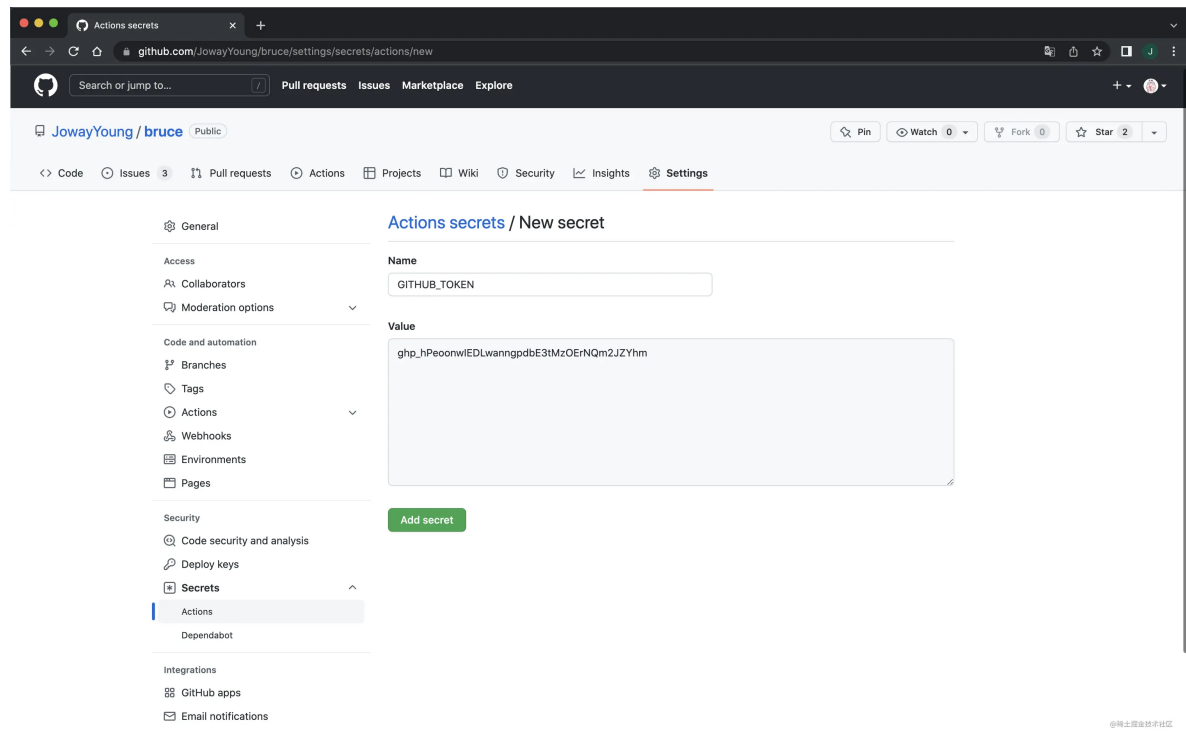
[Delete](#)

Expires on **Tue, Jun 7 2022**.

Personal access tokens function like ordinary OAuth access tokens. They can be used instead of a password for Git over HTTPS, or can be used to [authenticate to the API over Basic Authentication](#). @稀土掘金技术社区

最后创建**Actions Secret**。打开项目仓库地址，点击 **Settings**，再点击 **Secrets** 的 **Actions**，再点击 **New repository secret**。

在 **Name** 中输入密钥名称，例如 **GITHUB_TOKEN**。在 **Value** 中粘贴上述创建的 **GA_TOKEN** 的内容。



确定无误后点击 **Add secret**，生成一个 **Actions Secret**。



在配置文件中可通过 `{{ secrets.xyz }}` 的形式读取，**xyz** 为在 **Name** 中输入的 密钥名称。

gh-pages分支

Github仓库 创建 **gh-pages** 分支后会自动部署 **Github Pages** 上, 所以利用该特性把生成的静态文件提交到 **gh-pages** 分支上就可通过 **Github Pages** 自动生成静态站点。

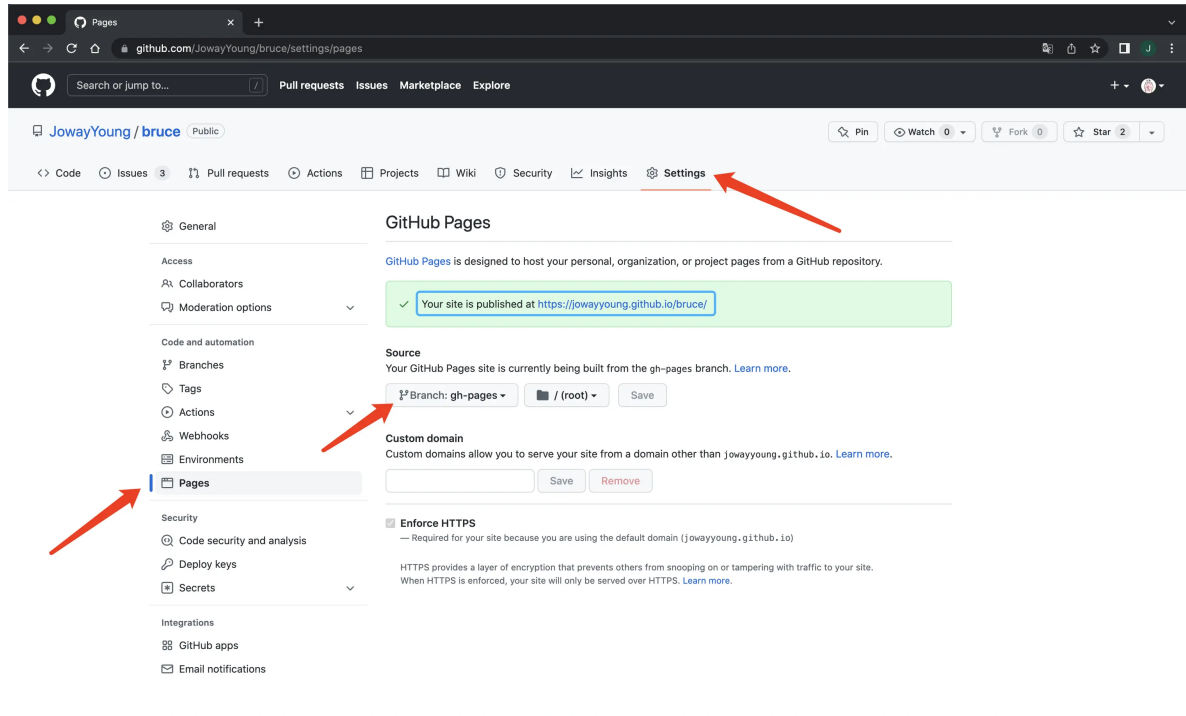
当然该操作无需手动，因为有相关第三方 **Actions** 可完成，后续编写 **配置文件** 会讲述。为了让 **Github Pages** 生效，还需在当前仓库配置。

打开项目仓库地址，点击 **Settings**，再点击 **Pages**，再点击 **Branch**，选择 **gh-pages**。会有 **root** 与 **docs** 两个文件夹可选，若把 **文档站点** 的静态文件放到 **docs** 文件夹，则选择 **docs**，否则全部情况选择 **root**。

点击 **Save** , 就会通过 **Github Pages** 自动生成静态站点了。站点的链接结构为以下形式。

`https://<username>.github.io/<project>`

txt



最终**bruce**的静态站点是**jowayyoung.github.io/bruce/**。

配置文件

根据手动部署的流程编写一份 **Github Actions** 配置文件。在根目录中创建 **.github/workflow** 目录, 进入目录创建 **cbd.yml** 文件, 加入以下内容。

ym1

```
name: CBD
on:
  push:
    branches:
      - main
jobs:
  cbd:
    name: doc cbd
    runs-on: ubuntu-latest
    steps:
      - name: Checkout
        uses: actions/checkout@v2
      - name: Build
```

```

    run: yarn && yarn run deploy
  - name: DeployGP
    uses: peaceiris/actions-gh-pages@v3
    with:
      github_token: ${ secrets.GITHUB_TOKEN }
      publish_dir: ./dist
  - name: DeployECS
    uses: easingthemes/ssh-deploy@v2
    env:
      ARGS: "-avz --delete"
      SSH_PRIVATE_KEY: ${ secrets.ECS_PRIVATE_KEY }
      REMOTE_HOST: ${ secrets.ECS_SERVER_HOST }
      REMOTE_USER: ${ secrets.ECS_REMOTE_USER }
      SOURCE: "dist"
      TARGET: "/www/static/doc/bruce"

```

整个 CI 操作 只有一个任务 `cbd` , `cbd` 由以下步骤组成。

- ✓ **Checkout**: 检出代码, 使仓库代码保持最新的状态
- ✓ **Build**: 打包文件, 先执行 `yarn` 安装依赖, 再执行 `yarn run deploy` 打包文件
- ✓ **DeployGP**: 部署文件到 `Github Pages`
- ✓ **DeployECS**: 部署文件到 阿里云的 ECS 云服务器

`checkout` 是一个官方 `Actions` , 用于自动检出最新代码。

`actions-gh-pages` 是一个第三方 `Actions` , 用于自动部署代码到 `Github Pages` 。
入参使用 `with` 配置, 可查看 [配置列表](#)。

第21章开发的 `文档站点` , 执行 `yarn run deploy` 会生成一个 `dist` 文件夹, 该文件夹存放着整个站点的静态文件, 所以 `publish_dir` 设置为 `./dist` 。

`ssh-deploy` 是一个第三方 `Actions` , 用于自动部署代码到服务器。入参使用 `env` 配置, 可查看 [配置列表](#)。

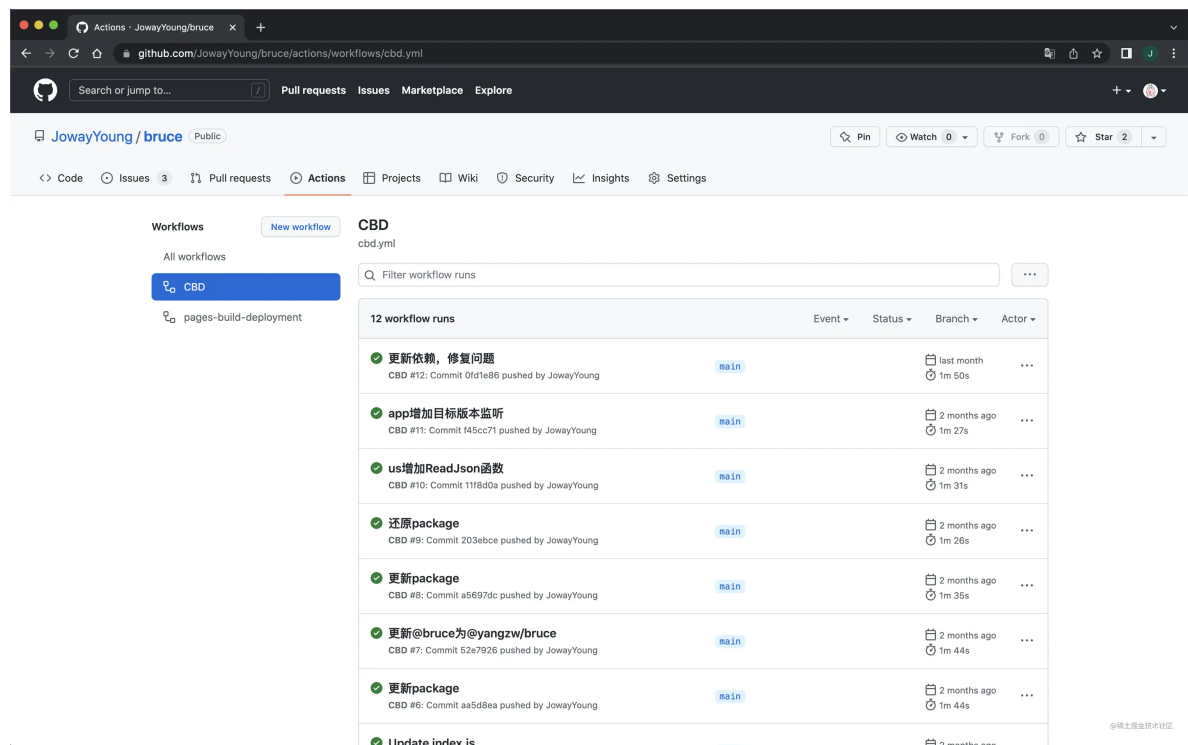
- **SSH_PRIVATE_KEY**: SSH 密钥, 登录服务器后可执行 `ssh-keygen -m PEM -t rsa -b 4096` 生成
- **REMOTE_HOST**: 服务器主机
- **REMOTE_USER**: 服务器用户

上述三个入参都可通过上述创建 `Actions Secret` 的方式生成。

- **SSH_PRIVATE_KEY** 指向 `secrets.ECS_PRIVATE_KEY`

- `REMOTE_HOST` 指向 `secrets.ECS_SERVER_HOST`
- `REMOTE_USER` 指向 `secrets.ECS_REMOTE_USER`

到此全部步骤完成，每次将代码提交到仓库，就会自动触发 `cbd.yml` 配置的 **CI操作** 自动执行。打开项目仓库地址，点击 **Actions**，就能看到全部 **Workflows**，点击 **CBD**，就能看到每次提交代码后自动执行的 **CI操作** 了。



总结

上述操作将手动部署流程转换为自动部署流程，还可尝试接入其他 **Actions** 操作其他工作流程，当然这些还是交给你主动去探讨。本章完毕就代表本课程已全部讲述完毕，终章就会与你一起总结本课程以及我对 **前端工程化** 的一些感悟与想法。

本章内容到此为止，希望能对你有所启发，欢迎你把自己的学习心得打到评论区！

- ☑ 示例项目：[fe-engineering](#)
- ☑ 正式项目：[bruce](#)

留言

输入评论（Enter换行，Ctrl + Enter发送）

发表评论

全部评论 (5)

小萌新入场



IT

22天前

gh-pages分支 后面的 配置文件，看到这里我已经模糊了，在这之前都跟着操作了，页面链接也出来了的，接下来不会了 那个CBD 🤔，

👍 1 💬 回复

小萌新入场



IT

22天前

Github Token与Actions Secret, 在这个地方感觉不是太清楚，这一步骤==》最后创建Actions Secret。怎么创建的，我是看着下面的配图，进了自己的库里面找到的，因为配图找到的，没配图都不知道怎么操作，感觉断层了

👍 1 💬 回复

小萌新入场



IT

22天前

Github Actions 的配置文件下面，路径 .github/workflows
、.github/workflows/<name>.yml 是 .github/xxxx 吧

👍 1 💬 回复



getify



前端 @ 前端。。。 2月前

持续集成又称Continuous Integration，简称CI，指多名开发者在开发不同代码过程中可频繁地将代码合且互相不影响工作。

将代码合并？这句话不太通顺

👍 点赞 💬 1



JowayYoung



(作者)

2月前

是的，将代码合并，漏了个“并”字

👍 点赞 💬 回复

